Quantum Mirror

Vadim Truhanov vt24011

Game description

The screen is split into two near-identical versions of each level, and the player acts as a qubit: |0> puts you in the upper level, |1> in the lower, and superposition puts you in both at once, with one set of controls moving both copies.

You collect limited-use quantum gates scattered through the level. H splits you into superposition and merges you back. Z flips your phase, which changes which level you exit into. X swaps you between levels when you’re not in superposition. Observers like surveillance cameras measure you and collapse you randomly into one level. Everything else is deterministic.

To win, you reach the exit. If both levels have an exit, both copies must reach it. If only one does, you must collapse into the right phase first. Dying (for example on electric blocks) or getting stuck restarts the level.

P.S.: Extended description in the README.md file.

Implemented Gameplay Mechanics

Basic:

- Movement (sideways movement + jumping)
- Power Sources (levers and pressure plates, both single and multi use)
- Power Users (doors that open up on being powered)
- Hazards (electric blocks that kill the player)
- Picking up and using gates
- Exit

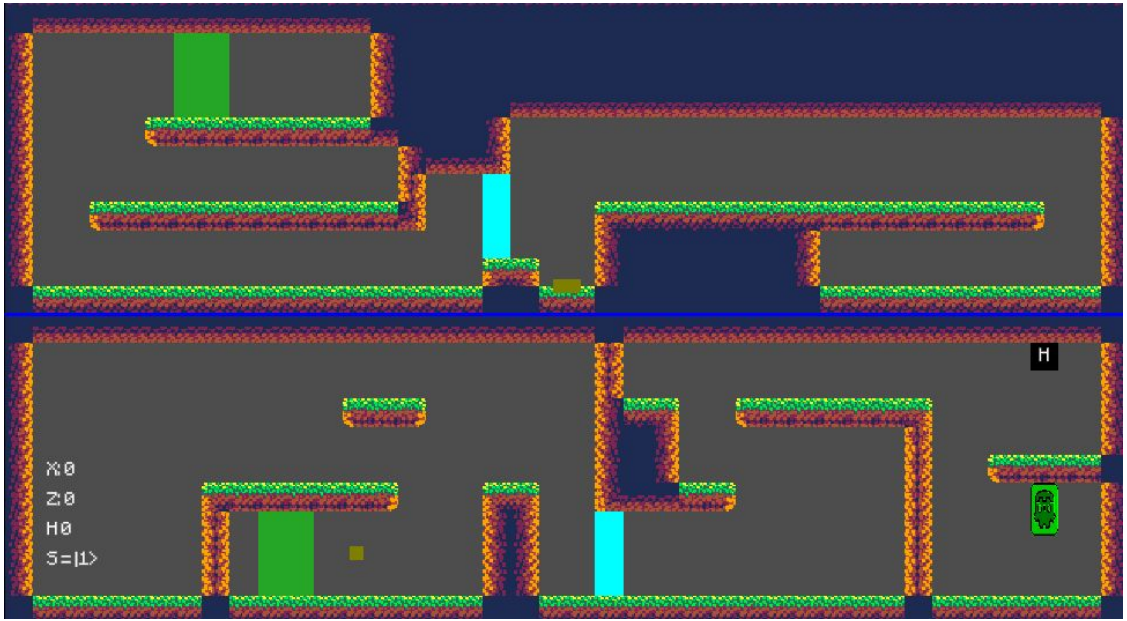
Quantum: Whole qubit quantum logic. This includes:

- Having single or double copies of player character in both versions of levels depending on qubit state
- X, Z and H gates logic
- Observation logic collapsing superposition randomly

How Quantum Mechanics affect the gameplay?

The game is based on qubit logic. Solving the **level is impossible without using quantum gates** and going back and forth between both versions of the levels.

Why? Because **both versions differ** in such ways that one level version helps another one e.g. a pressure plate that doesn't exist in the other version.



Todo list

Gameplay mechanics are nearly fully implemented. Mostly remaining tasks are from presentation department. Tasks are:

- Adding 3-4 more **levels**.
- Making and adding **sprites** to player character, tilemap, gates, doors and other gameplay objects / elements.
- Adding **hints** on how to play.
- Adding SFX, Music and some visual effect to enhance **player's experience**.
- Adding **interference** mechanic that would activate gates periodically adding more complexity into the level, trying to time everything else between interference activates.

Contribution and AI Usage

This is a solo project, fully developed by me. No AI tools were or will be used in its development; all code and assets are my own work.

Thanks for your attention!